10. RESULTS & SCREENSHOTS

This section summarizes the working modules, successful operations, and the final functional state of the MERN-based ride-hailing web application. It also provides visual evidence of the system's interfaces and error-handling capabilities.

10.1 Working Modules

The application is composed of several fully integrated, working modules across the stack:

- **Backend (API):** Modular endpoints successfully managing auth, users, drivers, rides, pricing, earnings, ratings, verification, analytics, and admin logic.
- **Frontend:** Dedicated role-based pages and components for Auth (login/register), Passenger (request rides), Driver (accept/complete rides), and Admin (dashboard, user/driver management), along with shared UI components like GeocodingAutocomplete.
- **Realtime:** Socket modules (sockets/) that successfully enable driver-passenger matching and live ride updates without page reloads.
- **Utilities & Services:** Functional utility scripts for fare calculation, distance computation, mail.service for emails, token utilities for JWT generation, and upload middleware for handling verification documents.

10.2 Successful Operations

The system successfully executes the following core operations:

- **User Registration & Login:** New users can securely register and obtain JWT access tokens via the authentication endpoints.
- **Driver Verification:** Drivers can upload compliance documents, which flow into an admin queue for approval via the verification endpoints and uploads directory.
- **Complete Ride Lifecycle:** Passengers can create a ride request, which is successfully broadcasted to nearby drivers. Once a driver accepts, the ride state updates (arrived, start, end) are stored in the database and emitted live via WebSockets.

- **Pricing Calculation:** The system dynamically calculates fares using the pricing configuration and fare utilities.

10.3 Final Functioning of the Web App

The final functioning web app operates as a cohesive, real-time ecosystem. It effectively bridges the gap between passengers seeking rides and drivers providing them, all under the supervision of a centralized admin panel. The integration of React, Node.js, MongoDB, and Socket.IO ensures that when a passenger books a ride, the nearest driver receives an instant popup, and the subsequent trip is tracked seamlessly. The role-based access control strictly enforces security, ensuring that passengers, drivers, and admins only interact with data pertinent to their operations.

10.4 Screen Layouts & Operations

1. Homepage (Booking Interface)

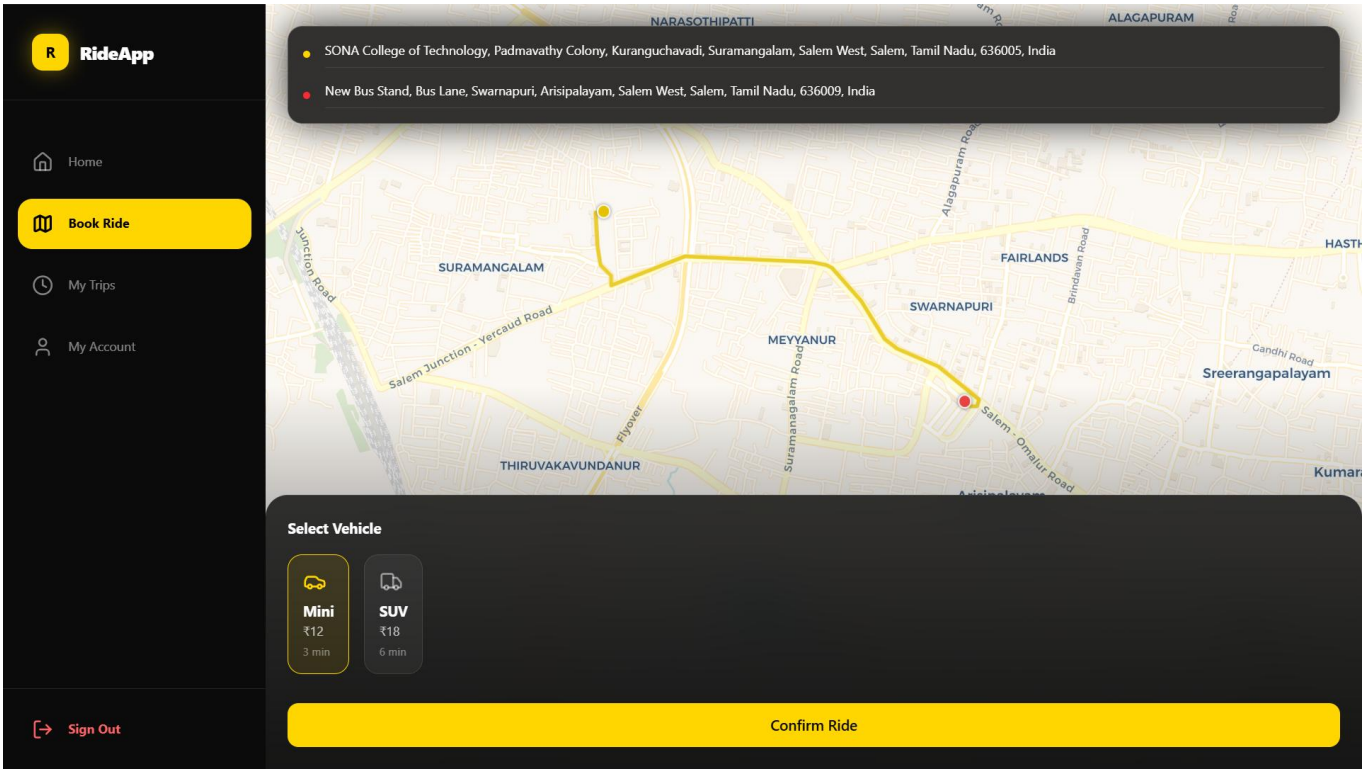
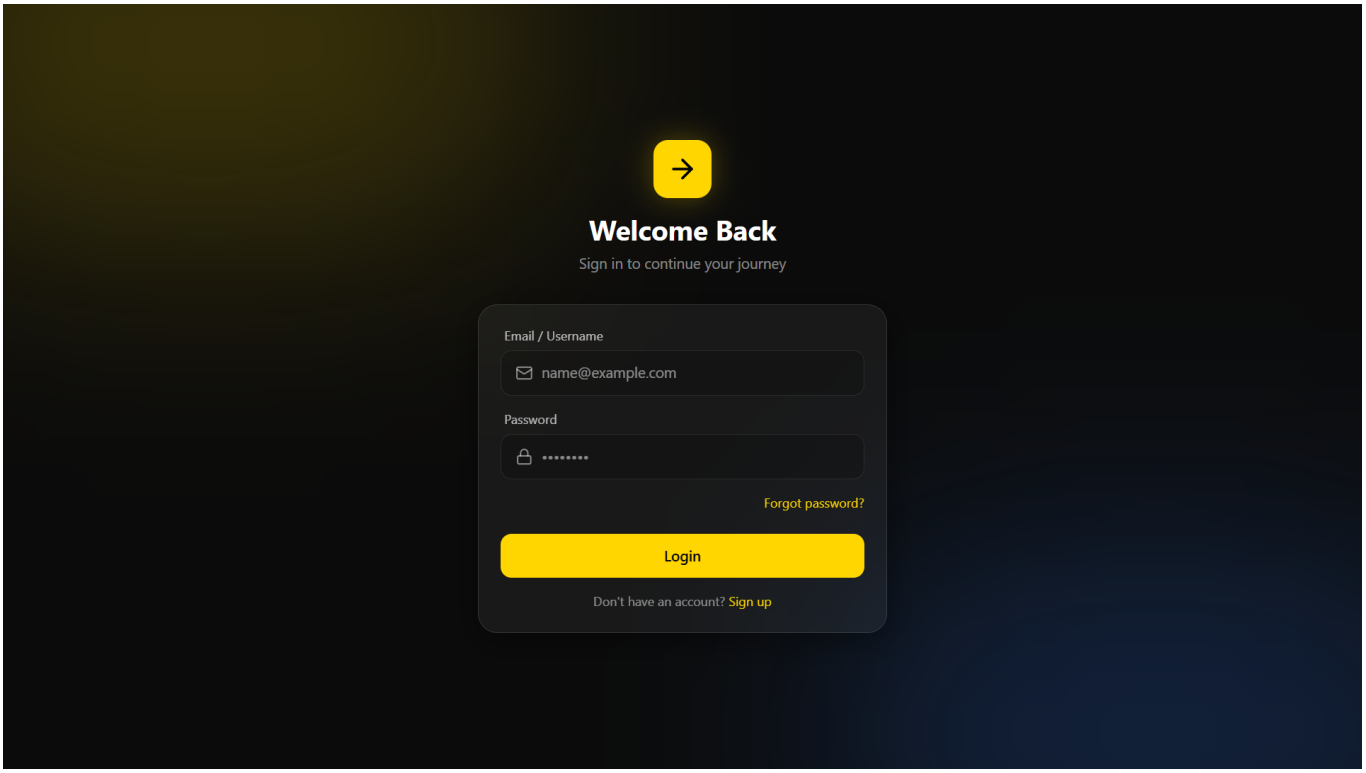


Figure 10.1: The primary booking screen showing the map interface and location input.

2. Login / Register



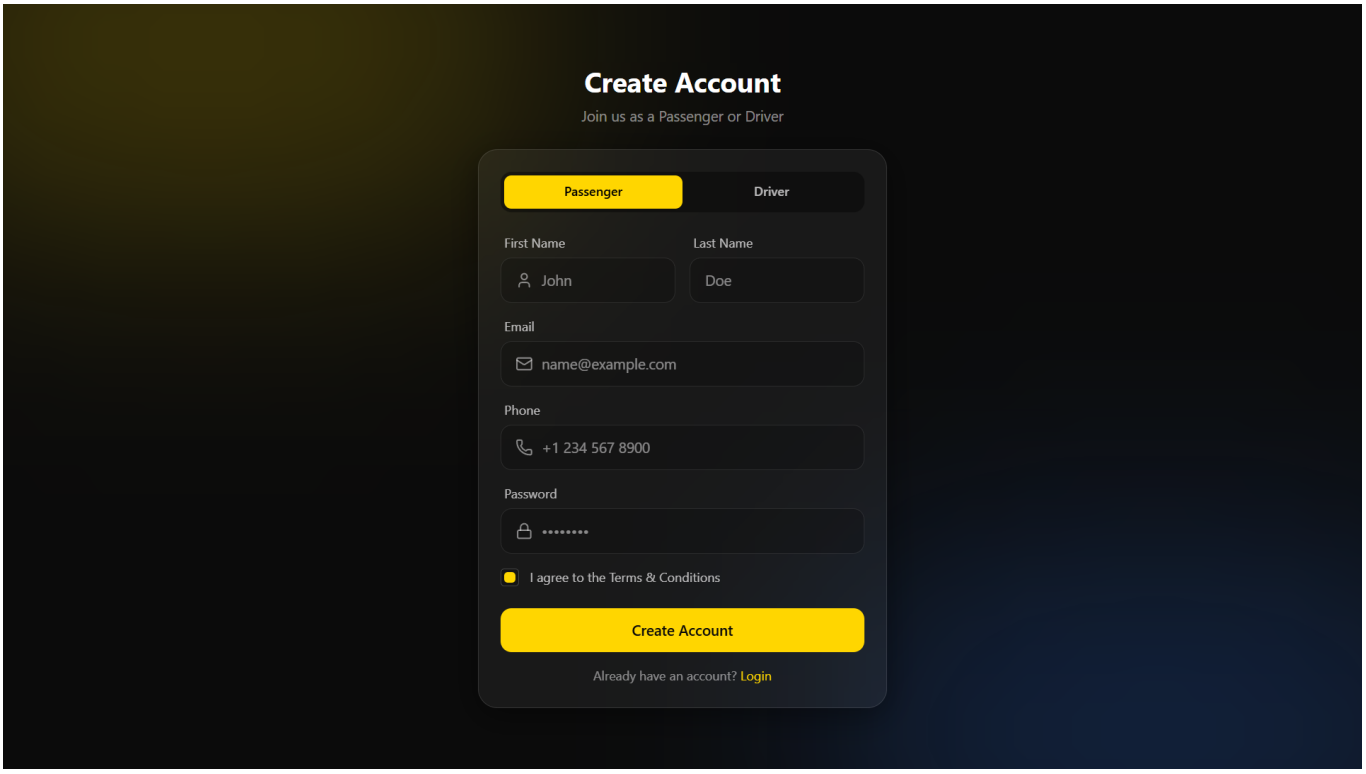


Figure 10.2: The authentication interface allowing users to log in or register with role selection.

3. Dashboard

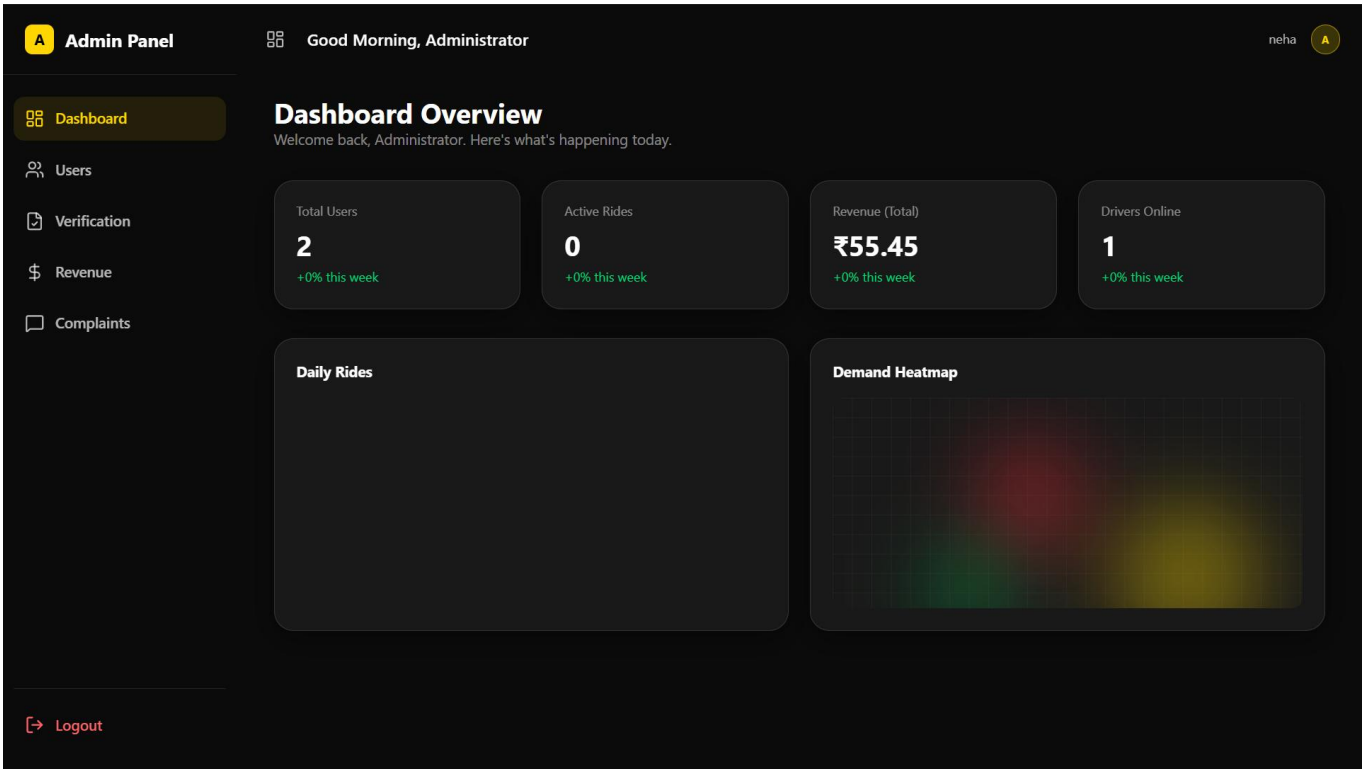


Figure 10.3: The Admin Dashboard displaying system KPIs, revenue, and active ride statistics.

4. CRUD Pages (User/Driver Management)

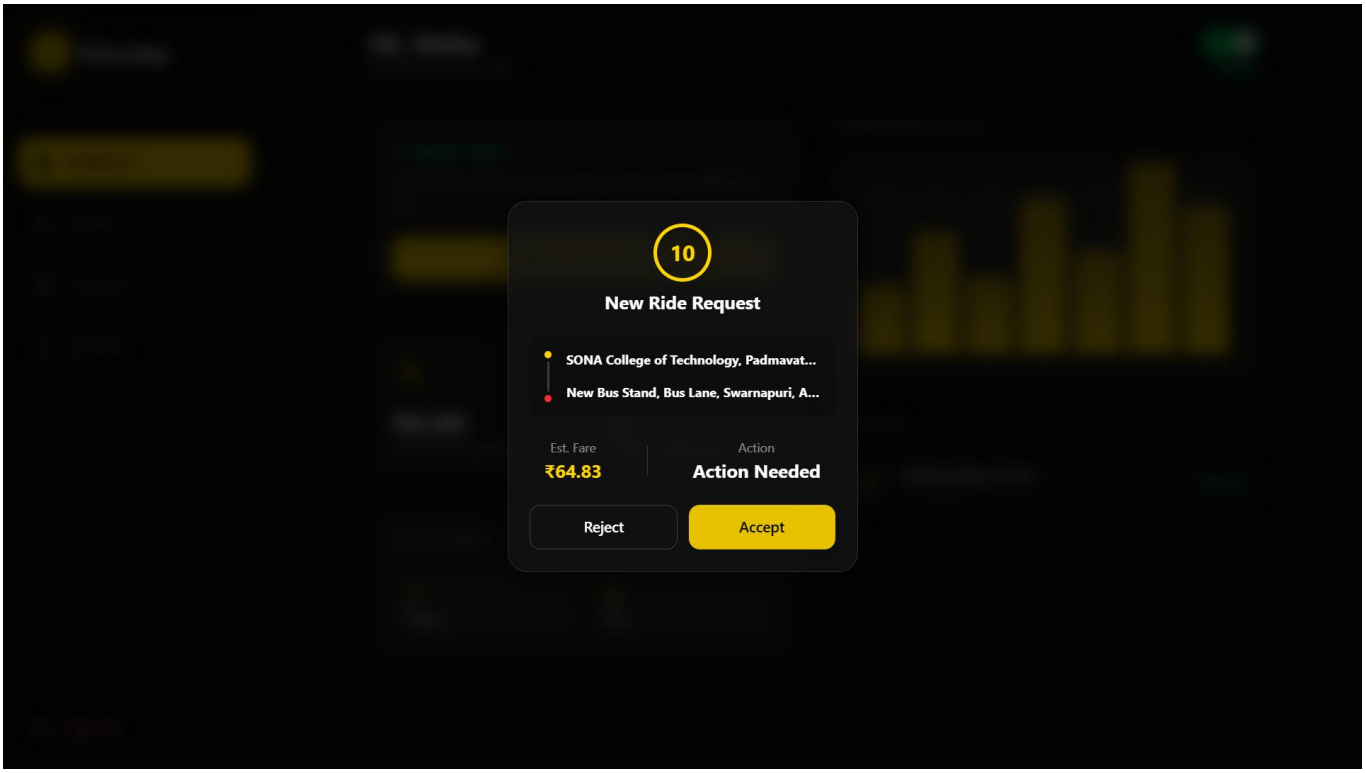


Fig 10.4 - a). Ride Request – from passenger

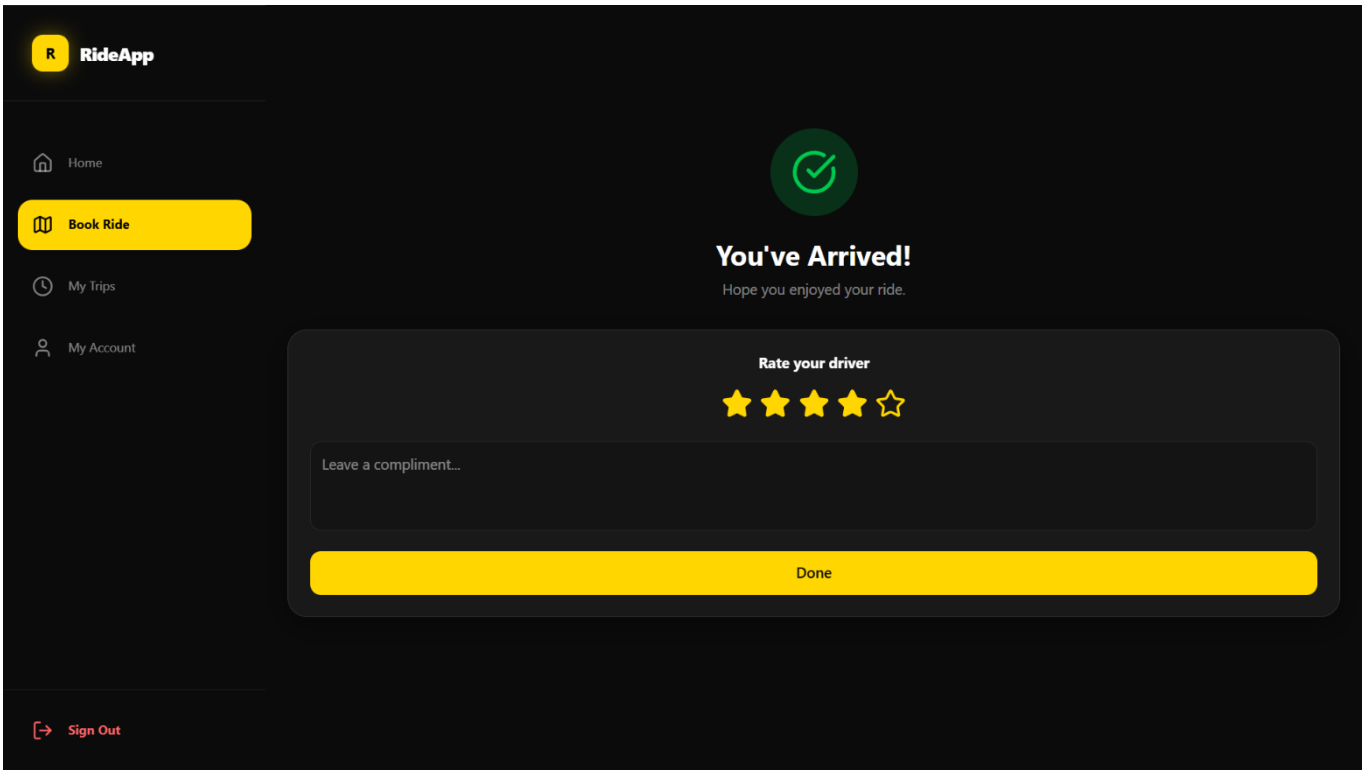


Fig 10.4 - b) Ride Completed and Rating Screen

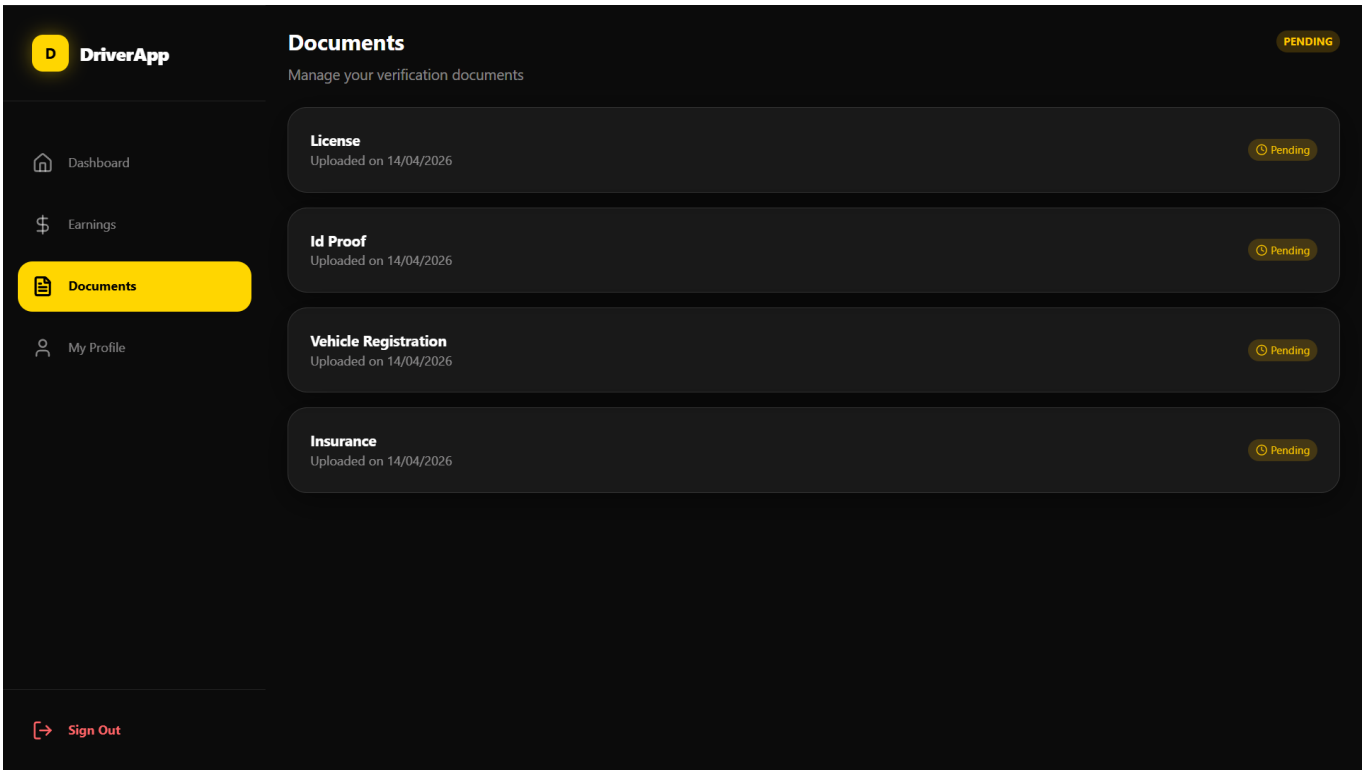


Fig 10.4 - c) Driver Verification – Pending status

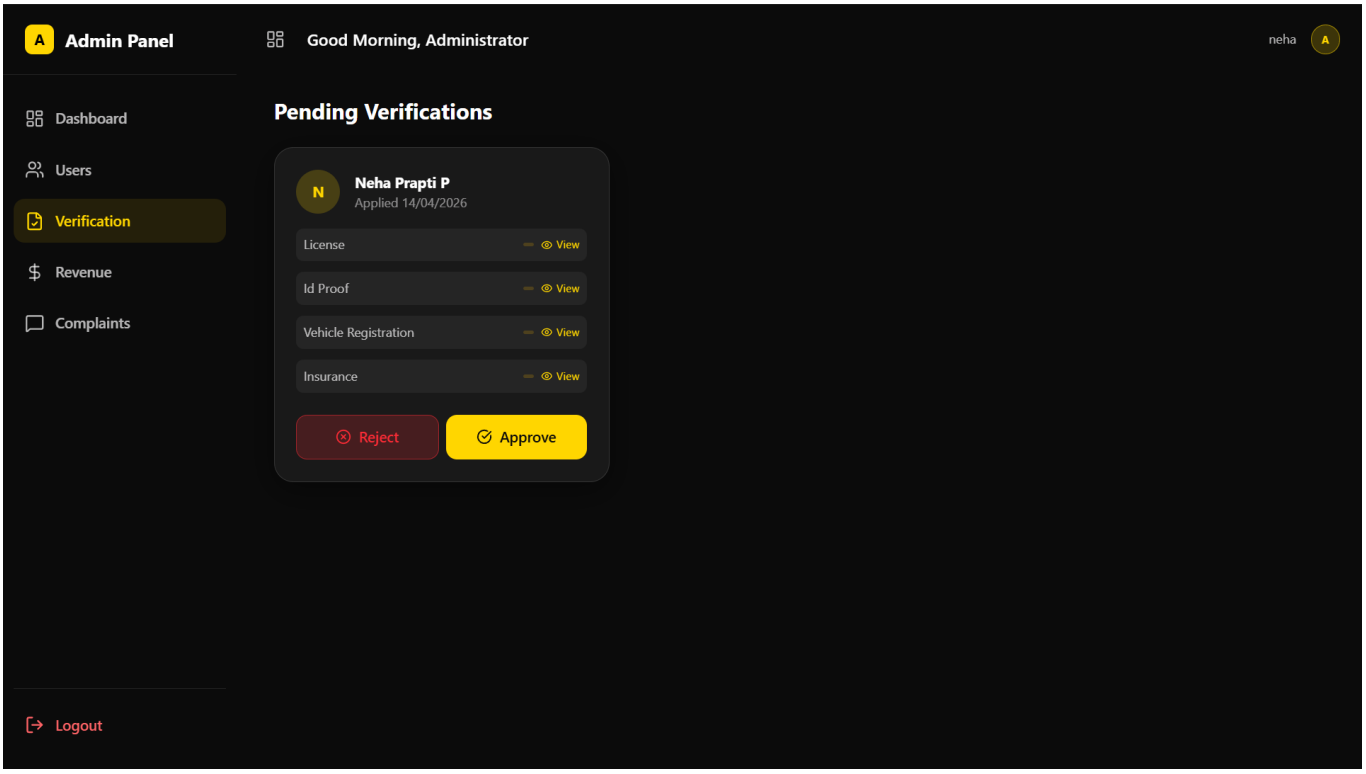


Fig 10.4 - d) Approving Section in Admin Panel

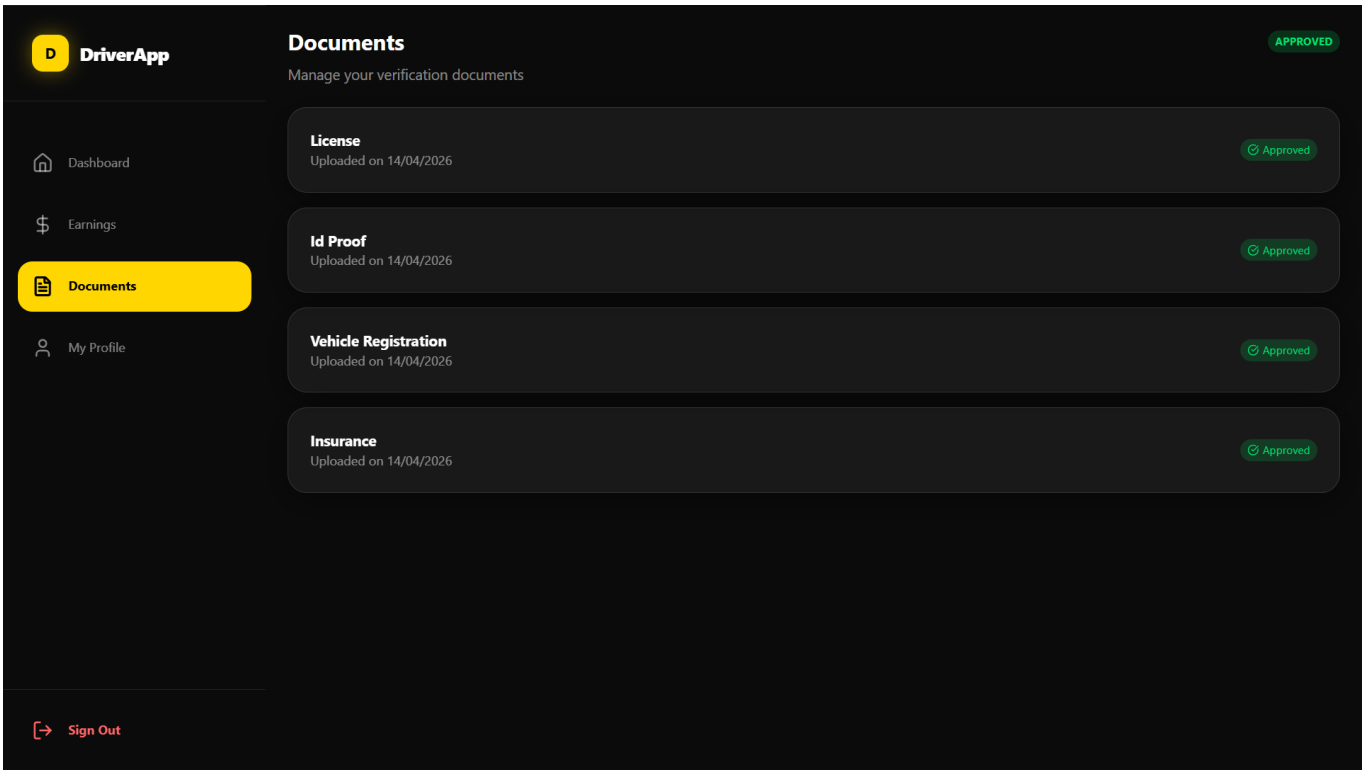


Fig 10.4 – e) Approved Screen in Driver Panel

Figure 10.4: Admin interface demonstrating Read, Update, and Delete operations for managing users and driver verifications.

5. Output Pages

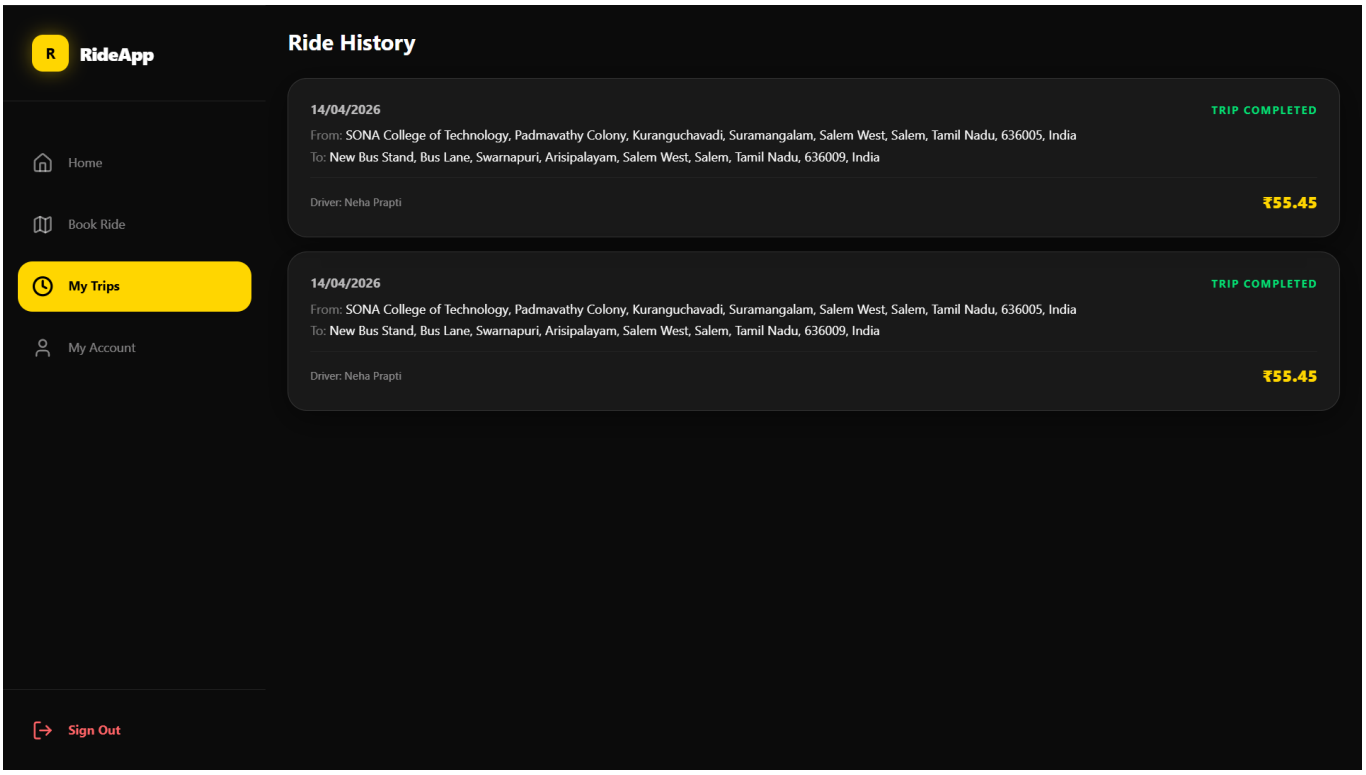


Figure 10.5: The final ride output screen showing trip details, fare breakdown

6. Error Handling & Validation

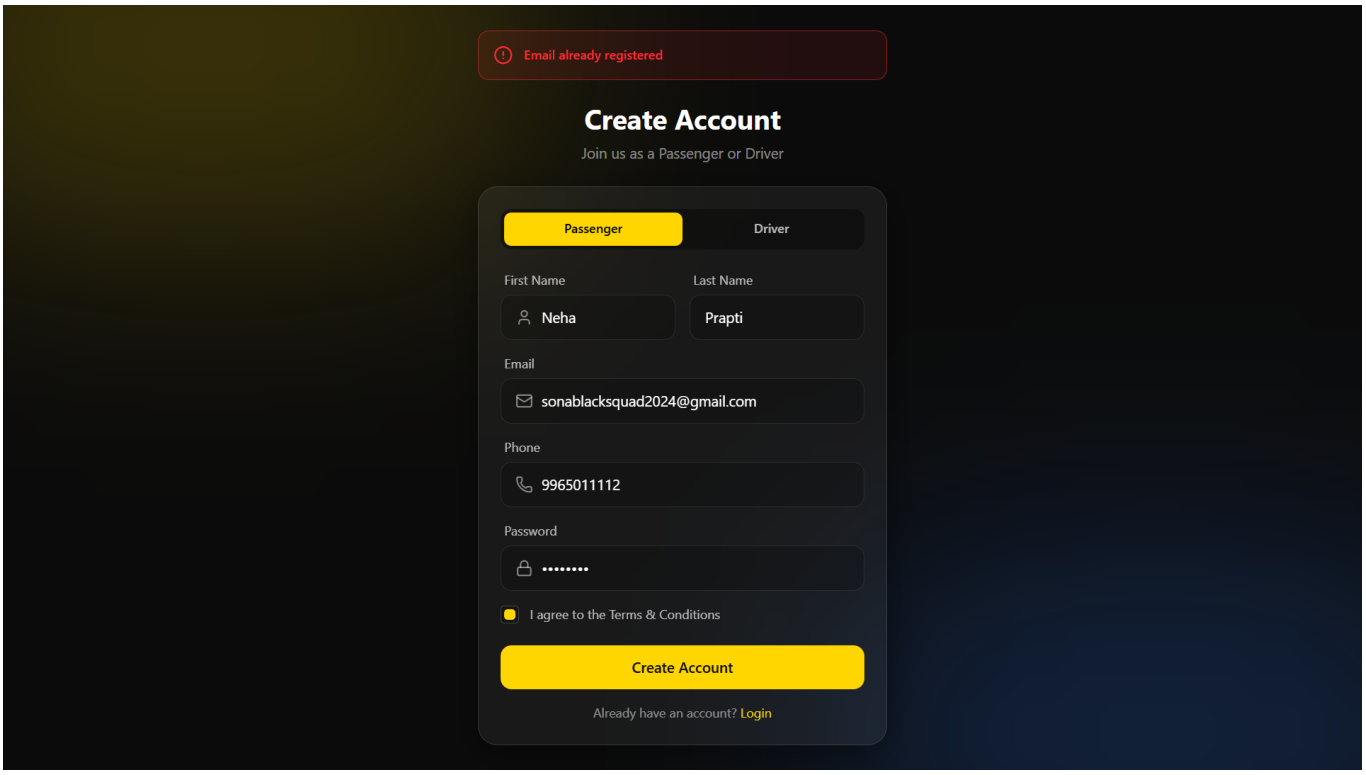


Figure 10.6: System demonstrating error handling